SIMATS ENGINEERING

OPERATING SYSTEMS

ASSIGNMENT – CO4

OPTIMISING FILE STORAGE AND DISK ACCESS

Course: Operating Systems (CSA04)

Course Outcome: CO4 – Optimise various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance (file systems, file allocation methods, modern file systems, distributed and cloud-based storage, disk scheduling and device management)

Nature of Assignment: Individual

Duration: One Week

Total Marks: 50

Submission Date:

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named CSA04-<RegisterNumber>-Assignment4 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.

• Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/192524372simats/CSA0413-OS

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Task 1 – File-System Design Review (6 Marks)

Scenario. One flat directory for all 12,000 students, file names chosen freely by students, contiguous allocation for every file, and no access control.

Task 1.1 – Evaluate the Proposal (2 Marks)

Seven weaknesses were identified (five were required). The search-time claim was measured with dirbench.py (Appendix A).

Weakness	Why it is a problem	Consequence
1. Name clashes	12,000 students choose their own names, so many will pick report.docx, a1.c, notes.txt. A directory can hold only one entry per name, so the second student either fails or silently overwrites the first.	Lost work, disputes, no way to tell whose file is whose.
2. Search time	An unindexed flat directory is searched linearly: on average n/2 = 6,000 and in the worst case 12,000 comparisons. Measured: **160.9 µs** per lookup for a linear scan against 0.066 µs for a hashed directory (about 2,400 times slower). Listing the directory returns all 12,000 entries (3.45 ms measured) mixed together. Note: a modern ext4 directory is hash-tree indexed, so a real stat() was fast (1.47 µs) – the indexing hides the lookup cost, but the other problems remain.	O(n) lookup, unusable listings, slow open as the user count grows.
3. File growth under contiguous allocation	A contiguous file needs all blocks in one run. If the neighbouring blocks are taken, the file cannot grow: the whole file must be copied to a bigger hole (O(file size)), or space must be pre-allocated and wasted. External fragmentation makes it worse: Task 2 shows 21 free blocks that cannot hold a 10-block file.	Failed appends, wasted space, periodic compaction.
4. Protection and privacy	With no access control any student can read, edit or delete anyone's files (assignments, marks, personal data). "Students will not look" is an assumption, not a control. An accidental rm * or a malicious student affects everyone.	Plagiarism, data leaks, breach of the college's duty to protect student data.

Weakness	Why it is a problem	Consequence
5. Backup and recovery	One flat volume means all-or-nothing backups; restoring one student's file requires searching or restoring everything. There is no per-user quota, versioning or snapshot, and one damaged directory or one server failure loses all 12,000 students' files.	Long recovery time, single point of failure.
6. Sharing for group projects	Without owners, groups or permissions, sharing means copying files (the copies diverge) or giving everybody access to everything.	Version conflicts or no safe collaboration.
7. No quotas or accounting	One student storing a 50 GB video fills the shared pool for everyone; the administrator cannot see who uses what.	Denial of service by accident.

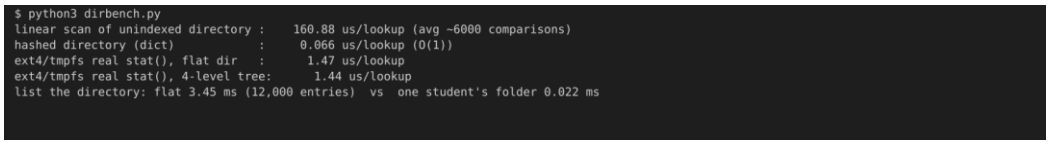


Figure 1.1 – Measured directory look-up cost (dirbench.py)

Task 1.2 – Improve the Design (3 Marks)

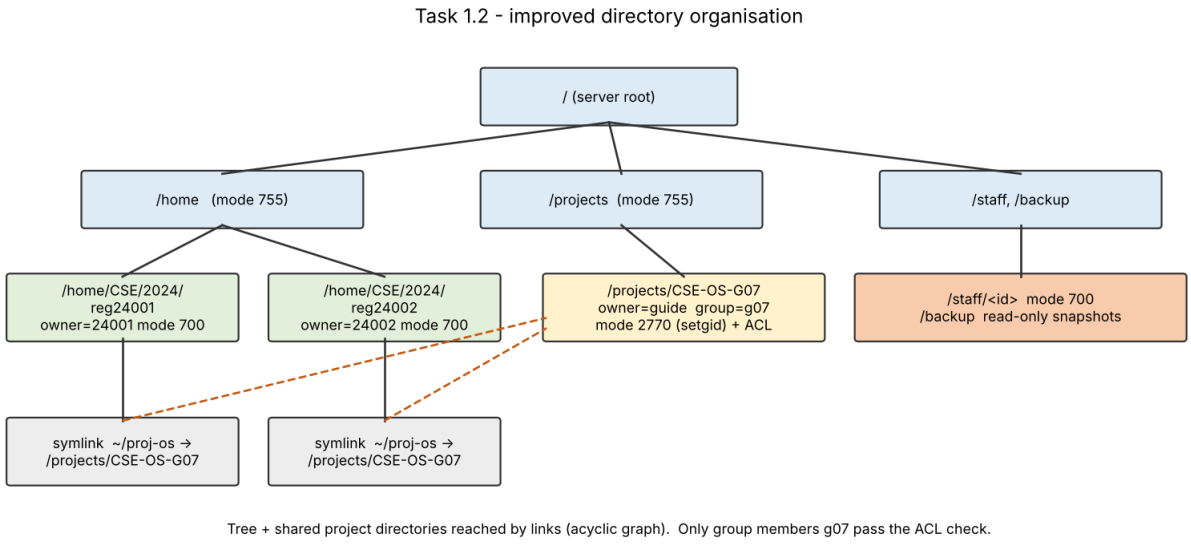


Figure 1.2 – Proposed directory structure (tree with links = acyclic graph)

(a) Directory structure

A **tree** of home directories, extended into an **acyclic graph** by links to shared project directories: /home/<dept>/<year>/<regno> for each student (about 100 entries per folder instead of 12,000), /projects/<course>-<group> for group work, /staff/<id> for

staff, and /backup for read-only snapshots. Lookup cost becomes O(depth) = 4 components, each resolved through an indexed (hashed or tree) directory, so the cost no longer depends on the number of students.

(b) Naming rules

- The folder name is the register number, which is unique – so clashes between students are impossible; inside a folder the student may use any names.
- Allowed characters: letters, digits, -, _, .; maximum 255 bytes per name; no spaces or leading dots; case-sensitive.
- Group projects: <course>-<group> (for example CSA04-G07); recommended file name pattern <course>_<regno>_<topic>_v<n>.<ext> for submissions.

(c) File attributes kept in the inode

Attribute	Purpose
Owner (UID), group (GID)	Who owns the file and which group may share it
Permission bits + ACL	rwx for owner, group, others; extra entries for named users
Size, block/extent pointers	Where the data is (indexed/extent allocation – not contiguous)
Timestamps (atime, mtime, ctime)	Auditing, backup (incremental backups use mtime)
Link count	Number of hard links; file is freed when it reaches 0
Quota accounting	Charges the file to its owner (e.g. 5 GB per student)

(d) Access control and shared folders

```
# private home directory: only the owner can enter
mkdir -p /home/CSE/2024/reg24001 && chown reg24001:students
/home/CSE/2024/reg24001
chmod 700 /home/CSE/2024/reg24001          # rwx------

# shared project folder: group members only, new files inherit the group (setgid)
groupadd g07 && usermod -aG g07 reg24001 && usermod -aG g07 reg24002
mkdir -p /projects/CSA04-G07 && chown guide:g07 /projects/CSA04-G07
chmod 2770 /projects/CSA04-G07             # rwxrws---
setfacl -m u:reg24005:r-x /projects/CSA04-G07   # read-only access for one named
reviewer

# link from each member's home (a symbolic link = a second path to the same
directory)
```

```
ln -s /projects/CSA04-G07 /home/CSE/2024/reg24001/proj-os

# quota and backup
setquota -u reg24001 5G 6G 0 0 /home        # soft 5 GB, hard 6 GB
# nightly read-only snapshot of /home to /backup (LVM/ZFS/rsync --link-dest)
```

Allocation method. Replace contiguous allocation with indexed/extent-based allocation (as in ext4 or XFS): files can grow wherever free blocks exist, there is no external fragmentation problem, and direct access stays fast (Task 2).

Task 1.3 – Design Defence (1 Mark, ≤100 words)

> Every student owns a private home directory with mode 700, so only the owner and the administrator can open it; nobody can list or read another student's files. Sharing is explicit: a project folder is owned by the guide, limited to a group of members through group ID and ACL entries, and reached by a symbolic link. Access is therefore granted per group, never to everyone, and is withdrawn by removing the member from the group. Quotas, read-only snapshot backups and access logs add accountability. Privacy is the default; sharing is a deliberate, auditable exception.

(Word count: 95.)

Alternative Concept and Complexity Analysis – cloud storage (Google Drive / OneDrive)

Solution if the files lived in a cloud drive. Each student gets a drive with a folder tree of its own; the college administrator creates a shared drive per project, sets sharing rules in the admin console (no "anyone with the link"), assigns the groups, quotas and retention policy, and relies on the provider for replication and version history. The OS-level directory structure, permission bits and backups are replaced by the provider's service.

```
ALGORITHM OpenFile(path)                    -- server with its own directory tree
INPUT  path = /home/CSE/2024/reg24001/report.docx
OUTPUT file handle or error
BEGIN
   dir <- root
   FOR EACH component IN path
      entry <- IndexedLookup(dir, component)      -- O(1) hash / O(log n) B-tree
      IF entry = NOT FOUND THEN RETURN error
      IF NOT PermissionAllows(entry, user) THEN RETURN "access denied"
      dir <- entry
   ENDFOR
   RETURN open(dir)
END                                  -- total O(d), d = depth
```

```
ALGORITHM OpenFileCloud(name)                -- cloud drive
BEGIN
   token <- Authenticate(user)               -- network round trips
   id    <- ServerSideSearch(token, name)    -- O(log n) + one WAN round trip
   RETURN Download(id)                       -- RTT + size / bandwidth
END
```

Criterion	Shared server with tree (improved)	Cloud drive (Drive / OneDrive)
Time to find a file	O(d) path components, each O(1) (hash) or O(log n) (B-tree). Measured: 1.4 µs per stat() on the local file system. Over a campus LAN each component may cost about 0.5 ms, so about 2–3 ms in total.	Server-side index search is O(log n), but every request pays the wide-area round trip (typically 30–100 ms) plus authentication; about 100–500 ms to search, shorter if the file is already synced locally.
Time to open	LAN read at 1 Gbit/s: a 10 MB file takes about 80 ms.	Download time = RTT + size / bandwidth: a 10 MB file at 20 Mbit/s takes about 4 s, unless cached.
Storage used (including copies)	1 copy on RAID + 1 backup copy (about 2×). Space and quota are controlled by the college.	Provider keeps several hidden replicas (not charged), but the quota also counts versions and the recycle bin, and each synced device holds a full copy: n devices means n + 1 copies of the data.
Control over privacy	College owns the hardware, keys, ACLs and logs; data stays on the campus. Risk: the college must run security itself.	Provider stores and processes the data; control is limited to the admin console (sharing rules, DLP, retention, data location) and the contract. Risks: link-sharing mistakes, account take-over, foreign jurisdiction.
Space complexity of metadata	O(number of files) inodes on the server.	O(number of files × versions) in the provider's database.

Result: The flat, contiguous, unprotected proposal fails on name clashes, search time, growth, privacy, backup and sharing. A tree of private home directories with

> mode 700, group-shared project directories reached by links, extent-based allocation, quotas and snapshots removes each weakness. A cloud drive removes the administration work but gives up control over where the data lives and adds WAN latency.

Task 2 – From Workload to File-Allocation Method (8 Marks)

Scenario. A network video recorder stores footage from 32 cameras. Files grow continuously while recording, are read mainly from start to end, are sometimes searched by time (random access) and are deleted after 30 days.

Task 2.1 – Identify the Operations (2 Marks)

Assumption: each camera records at 4 Mbit/s (0.5 MB/s), a typical value for HD surveillance video.

Operation	Frequency (estimate)	Access pattern	What the file system needs
Create a new file (one per camera per hour, for example)	32 cameras × 24 files/day = 768 files/day	–	Fast allocation, low metadata cost
Append (continuous recording)	Constant: 32 × 0.5 MB/s = **16 MB/s** ≈ 1.38 TB/day, about **41.5 TB** in 30 days	Sequential write, append-only	Cheap growth, no relocation, large extents
Read for playback	Occasional, a few operators at a time	Sequential (start to end)	High sequential throughput, read-ahead
Seek by time (jump to 14:32)	Several per playback session	**Direct / random** (time → offset → block)	O(1) or O(log n) location of block k
Delete expired files	Once per file after 30 days (768 files/day)	Metadata operation	Fast free of large files, low fragmentation
Metadata (stat, list directory)	Low	Direct	Small directory

The workload is write-heavy and append-only with sequential reads and occasional direct access. The key requirements are cheap growth, good sequential throughput, a fast way to reach block k, and little fragmentation after many create/delete cycles.

Task 2.2 – Compare the Allocation Methods (2 Marks)

Criterion	Contiguous	Linked (FAT in memory)	Indexed (inode / extents)
Sequential access	Best: one seek, then a continuous read	Good if blocks are close together; otherwise one seek per block	Good; read-ahead works along the index
Direct access	O(1): address = start + k	Poor: O(k) pointer follows (plain linked list = k disk reads; FAT = k memory look-ups)	O(1) with a few index reads (2–4)
File growth	Poor: needs a free run after the file; otherwise copy or reserve space in advance	Excellent: take any free block	Excellent: take any free block, add a pointer or extent
Fragmentation	Severe external fragmentation, compaction needed	No external; internal only in the last block; files may scatter	No external; index blocks cost a little space; extents keep files mostly contiguous
Reliability	A lost directory entry loses the file; no pointers to corrupt	A lost pointer or FAT damage cuts the chain: all blocks after it are lost (FAT is critical)	Damage to the index loses only that one file; the index can be duplicated
Space overhead	None	4 bytes per block in the FAT (or a pointer inside each data block)	Index blocks, about 0.1%

Decision: indexed allocation with extents (as in ext4/XFS) fits this recorder best: appends are cheap, extents keep each recording nearly contiguous so sequential reads stay fast, and a time-based seek needs only O(1) lookups.

Task 2.3 – Calculate the Cost (3 Marks)

Given: file size 4 GB = 2^{32} bytes, block size 4 KB = 2^{12} bytes, so the file has $2^{32}/2^{12} = 2^{20}$ = **1,048,576 blocks**. The last block has number 1,048,575 (counting from 0). A block address is 4 bytes, so one 4 KB index block holds 4096/4 = **1024 pointers**.

Method	Reasoning	Disk reads for the last block
Contiguous	Directory gives start block s; address of the last block = s + 1,048,575, computed arithmetically.	**1** (+1 for the directory/inode)
Linked list (pointer in each block)	Block 0 is found from the directory; each block holds the address of the next, so to read the last block all 1,048,575 predecessors must be read first.	**1,048,576**
FAT (table in memory)	The chain is followed in the in-memory FAT: 1,048,575 memory look-ups, but only one disk read (the data block).	**1** disk read (+1,048,575 RAM look-ups)
Indexed – single-level index block	One index block holds only 1024 pointers (4 MB of data), so a 4 GB file does not fit; a multi-level index (Unix inode) is needed.	Not possible with one level
Indexed – Unix inode (12 direct, single, double, triple)	Direct: blocks 0–11. Single: blocks 12 – 1,035. Double: blocks 1,036 – 1,049,611. Block 1,048,575 falls in the **double-indirect** range: read inode (usually already in memory), the double-indirect block, one single-indirect block, then the data block.	**3** reads after the inode (**4** including the inode)

Maximum file size of the inode (12 direct, 1 single, 1 double, 1 triple; 4-byte addresses, 4 KB blocks)

Pointer type	Blocks addressed	Bytes addressed
12 direct	12	12 × 4 KB = 48 KB
1 single indirect	1024	1024 × 4 KB = 4 MB
1 double indirect	$1024^2 = 1{,}048{,}576$	4 GB
1 triple indirect	$1024^3 = 1{,}073{,}741{,}824$	4 TB
Total	$12 + 1024 + 1024^2 + 1024^3$ = **1,074,791,436** blocks	**4,402,345,721,856 bytes ≈ 4.004 TB** (4 TB + 4 GB + 4 MB + 48 KB)

> **Remark.** The 4 GB file fits exactly within the double-indirect range (12 + 1,024 + 1,048,576 = 1,049,612 blocks ≥ 1,048,576). ext4 replaces this block map by an extent tree: a 128 MB extent covers 32,768 blocks, so a 4 GB file needs only 32 extents and about 1–2 reads to reach any block.

Task 2.4 – Free-Space Management (1 Mark)

4 TB disk, 4 KB blocks: $2^{42}/2^{12} = 2^{30}$ = **1,073,741,824 blocks**.

Method	How it works / cost	Size for this disk
Bit vector	1 bit per block (1 = free). Finding a free block or a run of n free blocks is fast with word-wide operations; easy to find contiguous runs. The whole vector should be kept in memory (or in a two-level bitmap).	2^{30} bits = 2^{27} bytes = **128 MB**
Linked free list	Each free block stores the address of the next free block. No extra space, but finding n free blocks needs n disk reads; no contiguous-run search.	0 extra (uses the free blocks); head pointer only
Grouping	The first free block stores the addresses of n further free blocks (4096/4 = 1024 per block; the last points to the next group block). Gives many free blocks per read.	Overhead of 1 block per 1023 free blocks = 0.1% of the free space
Counting	Store (first free block, run length) pairs, 8 bytes each. Very compact when free space is in long runs, as with large video extents.	8 bytes per free run; tiny if runs are long

Choice for the recorder: a bit vector (or extent/counting tree as in XFS) because the video files are large and extent allocation needs long free runs. A 128 MB bit vector is small compared with 4 TB of storage (0.003%).

Complete the Code – first-fit contiguous allocation and indexed allocation

TODO 1 was implemented as a first-fit scan: count consecutive free blocks, reset the counter at every used block, and when the run reaches size mark the blocks and return the start. The rest of the program is unchanged.

```
int alloc_contiguous(char file, int size)
{
   int run = 0;
   for (int b = 0; b < BLOCKS; b++) {
      if (disk[b] == '.') {
         run++;
         if (run == size) {
            int start = b - size + 1;
            for (int i = start; i <= b; i++) disk[i] = file;
```

```
            return start;
        }
      } else {
        run = 0;
      }
    }
    return -1;
}
```

```
$ gcc -std=c11 -Wall -Wextra -o task2 task2.c
$ ./task2
AAAAAA........CCCCC.............
E (10 blocks) contiguous: start 19
E (10 blocks) indexed: index block 6, data blocks 7 8 9 10 11 12 13 29 30 31
AAAAAAeEEEEEEECCCCCEEEEEEEEEEEE
```

Figure 2.1 – Output of task2.c (identical to the expected output)

Reading the output. After A(6), B(8), C(5), D(7) are allocated and B and D are released, the disk is AAAAAA........CCCCC.............: two holes, blocks 6–13 (8 blocks) and blocks 19–31 (13 blocks), 21 free blocks in total. First fit needs one hole of at least 10 blocks, finds the 13-block hole and places E at block 19, exactly as in the expected output. The indexed call then allocates E a second time (index block 6, data blocks 7–13 and 29–31) to show how the two methods use the free space.

> **Note on the question wording.** In the exact run given in the assignment, a 10-block E does fit contiguously at block 19, because the larger hole has 13 blocks. The statement "cannot be stored contiguously although 21 blocks are free" becomes true as soon as the file is larger than the largest hole. To demonstrate this, the same program was run with E = 14 blocks (file task2_e14.c in the repository, only the size changed). The result is shown below.

```
AAAAAA........CCCCC.............
E (14 blocks) contiguous: start -1
E (14 blocks) indexed: index block 6, data blocks 7 8 9 10 11 12 13 19 20 21 22 23 24 25
AAAAAAeEEEEEEECCCCCEEEEEEE.....
```

Why E cannot be stored contiguously although 21 blocks are free. Contiguous allocation needs one run of E's size. The 21 free blocks are split into holes of 8 and 13 blocks, so a 14-block file finds no run long enough and alloc_contiguous returns -1. This is **external fragmentation**: enough free space in total, but not in one piece. Only compaction (moving A and C, which means reading and rewriting all their data) could create a run of 14 blocks.

How indexed allocation solves the problem. An indexed file needs one index block plus any free data blocks, wherever they are. For E = 14 blocks the program takes block 6 as the index block (lowercase e) and the 14 data blocks 7–13 and 19–25,

using both holes and leaving blocks 26–31 free. No compaction is needed and block k of E is found with one extra read of the index block. The price is one block of index overhead (1 of 15 blocks here) and, on a hard disk, extra seeks when the blocks are far apart.

Alternative Concept and Complexity Analysis – linked allocation with an in-memory FAT

Solution. Instead of an index block per file, a single File Allocation Table with one entry per disk block is kept in memory: FAT[b] holds the number of the block that follows block b (or END). The directory stores only the first block of each file.

```
ALGORITHM FindBlock_FAT(first, k)
INPUT  first block of the file, block number k
OUTPUT disk block that holds file block k
BEGIN
   b <- first
   FOR i <- 1 TO k
      b <- FAT[b]                 -- one memory look-up per step: O(k)
   ENDFOR
   RETURN b
END

ALGORITHM FindBlock_Index(index, k)
INPUT  index of the file (in memory), block number k
OUTPUT disk block that holds file block k
BEGIN
   RETURN index[k]                -- O(1); a multi-level index needs 2-4 reads
END
```

```
$ gcc -std=c11 -Wall -Wextra -O2 -o fat_vs_index fat_vs_index.c
$ ./fat_vs_index
block k     FAT steps      FAT time (us)  index (us)   disk reads (FAT in RAM / index)
1000        1000           7.9            0.0009       1 / 2-4
100000      100000         2537.4         0.0010       1 / 2-4
500000      500000         12562.3        0.0010       1 / 2-4
1048575     1048575        24984.9        0.0009       1 / 2-4
```

Figure 2.2 – Measured traversal of a 1,048,576-block FAT chain (shuffled = worst case)

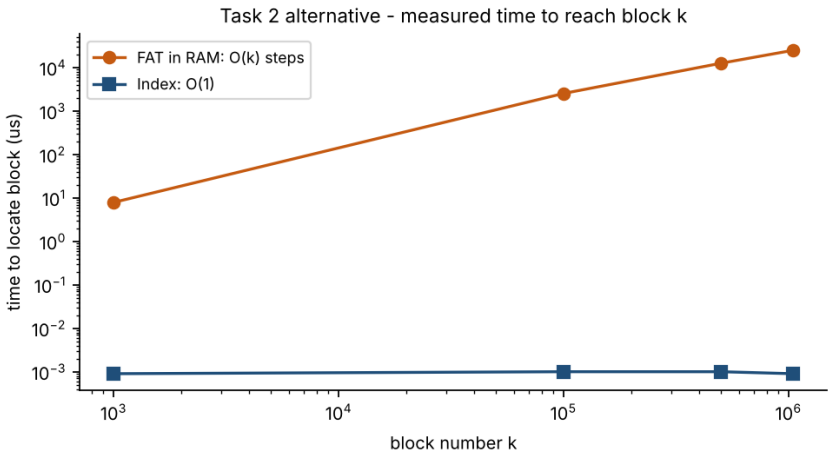


Figure 2.3 – Time to reach block k: FAT O(k) against index O(1)

Criterion	FAT (linked, table in memory)	Indexed (inode / extents)
Time to reach block k	O(k): k look-ups in the in-memory FAT. Measured for k = 1,048,575: **25.0 ms** (random layout) against 0.001 µs for the index. Disk reads: 1.	O(1) memory access once the index is cached; 2–4 disk reads for a multi-level index, 1 with extents.
Memory used for 4 TB disk	FAT must have one entry per block: 2^{30} entries × 4 bytes = **4 GB of RAM** (FAT32's 28-bit entries also cannot address 2^{30} clusters; larger clusters would be needed). Whole table is needed whether files are open or not.	Only the indexes of files that are open are kept in memory: tens of KB. Bit vector for free space: 128 MB.
Reliability if the table is damaged	Serious: the FAT holds all chains. One damaged entry cuts a file; a damaged FAT loses every file's structure (the data remain but cannot be put back in order). Mitigation: two copies of the FAT (FAT1/FAT2), checksum, scandisk.	Local: damage to one index block loses only that file. Journaling protects the metadata, and indexes may be duplicated.
Suitability for the recorder	Poor for time seeks in 4 GB files (25 ms of CPU per seek) and for memory at 4 TB.	**Better.** Direct access in O(1) and small memory.
Space complexity	O(disk blocks) = O(N)	O(blocks of open files)

> **Result:** Indexed (extent-based) allocation is the best fit for the 32-camera recorder: 1.38 TB/day of append-only writes, sequential reads and direct time seeks. For a 4 GB file the last block needs 3 reads after the inode (the double-indirect path), the inode supports at most about 4.004 TB per file, and the free-space bit vector of a 4 TB disk is 128 MB. The completed program reproduces the expected output exactly.

Task 3 – Modern File Systems (9 Marks)

Task 3.1 – Compare Modern File Systems (3 Marks)

File system	Max file size	Max volume size	Journaling	Snapshots	Checksums	Copy-on-write
FAT32	4 GiB − 1 byte	2 TiB (512-byte sectors); Windows formats only ≤ 32 GB	No	No	No	No
NTFS	≈ 8 PiB in practice (16 EiB theoretical)	256 TiB (64 KiB clusters); ≈ 8 PiB with 2 MiB clusters	Yes – metadata ($LogFile)	Via Volume Shadow Copy (VSS)	No (data)	No
ext4	16 TiB (4 KiB blocks)	1 EiB (≈ 16 TiB commonly used)	Yes – three modes (journal, ordered, writeback)	No native (use LVM)	Metadata and journal checksums only	No
XFS	8 EiB	8 EiB	Yes – metadata log	No native (LVM); reflink copies	Metadata CRC (v5 format)	Only reflink (shared extents), not full CoW
Btrfs	16 EiB	16 EiB	No journal; CoW plus a log tree for fsync	Yes – subvolume snapshots	Yes – data and metadata (crc32c, xxhash, sha256)	Yes

File system	Max file size	Max volume size	Journaling	Snapshots	Checksums	Copy-on-write
ZFS	16 EiB	256 ZiB	No journal; transactional CoW plus intent log (ZIL) for sync writes	Yes – cheap, writable clones	Yes – data and metadata (fletcher4, SHA-256)	Yes
APFS	≈ 8 EiB (64-bit)	≈ 8 EiB (container-limited)	No journal; CoW metadata	Yes	Metadata only	Yes (metadata, clones)

Note: limits are for common configurations (4 KiB blocks) and depend on the kernel or OS version; they should be checked against the vendor documentation when quoted in an exam.

Task 3.2 – Journaling and Consistency (2 Marks)

What happens when power fails during a write

Appending to a file changes several blocks: the data block, the inode (size, pointer or extent), the free-space bitmap and the directory time-stamps. The disk can write them in any order, so a power failure leaves **some but not all** of the updates on disk. Typical inconsistencies are: the bitmap says a block is used but no inode owns it (a leak); an inode points to a block that the bitmap still shows as free (it can be given to a second file – corruption); the inode size is updated but the data was never written (garbage or zeros). Without protection, fsck has to scan the whole disk after the crash, which takes minutes to hours on a multi-terabyte disk.

How journaling keeps the file system consistent

- **Write-ahead log:** the changes (descriptor + the new metadata blocks) are first written to a journal and made durable with a **commit record**.
- **Checkpoint:** only after the commit are the blocks written to their real places.
- **Recovery:** after a crash the kernel replays the committed transactions and ignores the incomplete one. Recovery time depends on the size of the journal (seconds), not on the size of the disk.

How copy-on-write keeps the file system consistent

A CoW file system (Btrfs, ZFS, APFS) never overwrites live blocks. A changed block is written to a new place, then its parent is rewritten to a new place, up to the root; finally one atomic write of the new root pointer (uberblock/superblock) commits

the whole tree. If power fails before that write, the old root and the old tree are intact, so the disk always shows a consistent, complete older version. There is nothing to replay: mounting is immediate. Snapshots are a by-product: keep the old root.

ext4 journaling modes

Mode	What is journaled	Behaviour after a crash	Speed / typical use
journal	Data **and** metadata are written to the journal first, then to their final place.	Safest: files are consistent and data is never stale.	Slowest: every data block is written twice. Used when data safety matters more than speed.
ordered (default)	Only metadata is journaled, but data blocks are forced to disk **before** their metadata is committed.	File system consistent; a file never points to unwritten data (new data may be lost, never garbage).	Good balance; default on most Linux distributions, a sound choice for general servers.
writeback	Only metadata is journaled; no ordering between data and metadata.	File system consistent, but a file may contain old or stale data after a crash.	Fastest; used for scratch data, or when the application (e.g. a database with its own log) enforces order.

Task 3.3 – Choose for Each Device (3 Marks)

Device	Choice	Justification
Pen drive used on Windows and Linux	**exFAT** (FAT32 for old devices)	Both systems read and write exFAT (Windows natively; Linux kernel 5.7+ natively, earlier through a package). It has no 4 GiB file limit (FAT32 has), a simple structure with no journal, which suits flash and removable media, and no ownership or permissions that would get in the way when the drive moves between computers. NTFS works but Linux support is weaker and its journal adds writes to flash. Use FAT32 only if the drive must work in old TVs or cameras and all files are under 4 GiB.

Device	Choice	Justification
Linux database server	**XFS** (ext4 with data=ordered is an equally acceptable alternative)	XFS is designed for large files and parallel I/O: allocation groups allow many threads to allocate at once, extents are large, the metadata journal gives fast recovery, and it scales to very large volumes. Mount with noatime and let the database use direct I/O. A copy-on-write file system is avoided for the data files because random page updates fragment (see the alternative analysis).
Laptop	The laptop's native file system: **ext4** (Linux), NTFS (Windows), APFS (macOS)	ext4 is stable, has low overhead, and its journal protects against power loss or battery run-out; extents and delayed allocation give good performance on SSD. On Windows, NTFS gives permissions, journaling, encryption and compression; on a Mac, APFS is designed for flash with snapshots and encryption. Btrfs is a good option on Linux if snapshots for rollback are wanted.
Backup server that needs snapshots	**ZFS** (Btrfs as the lighter alternative)	Instant, space-efficient snapshots (only changed blocks are stored), end-to-end checksums with automatic self-healing from redundancy, scrubs to detect silent corruption, built-in RAID-Z, compression and send/receive for replication. Each backup run becomes a snapshot that cannot be changed by ransomware on the clients.

Task 3.4 – Inspect a Real File System (1 Mark)

The commands were run on the Ubuntu 24.04.5 LTS machine used for this report (root file system /dev/vda, ext4). **Re-run them on your own machine and replace the screenshot** – the assignment asks for your own system.

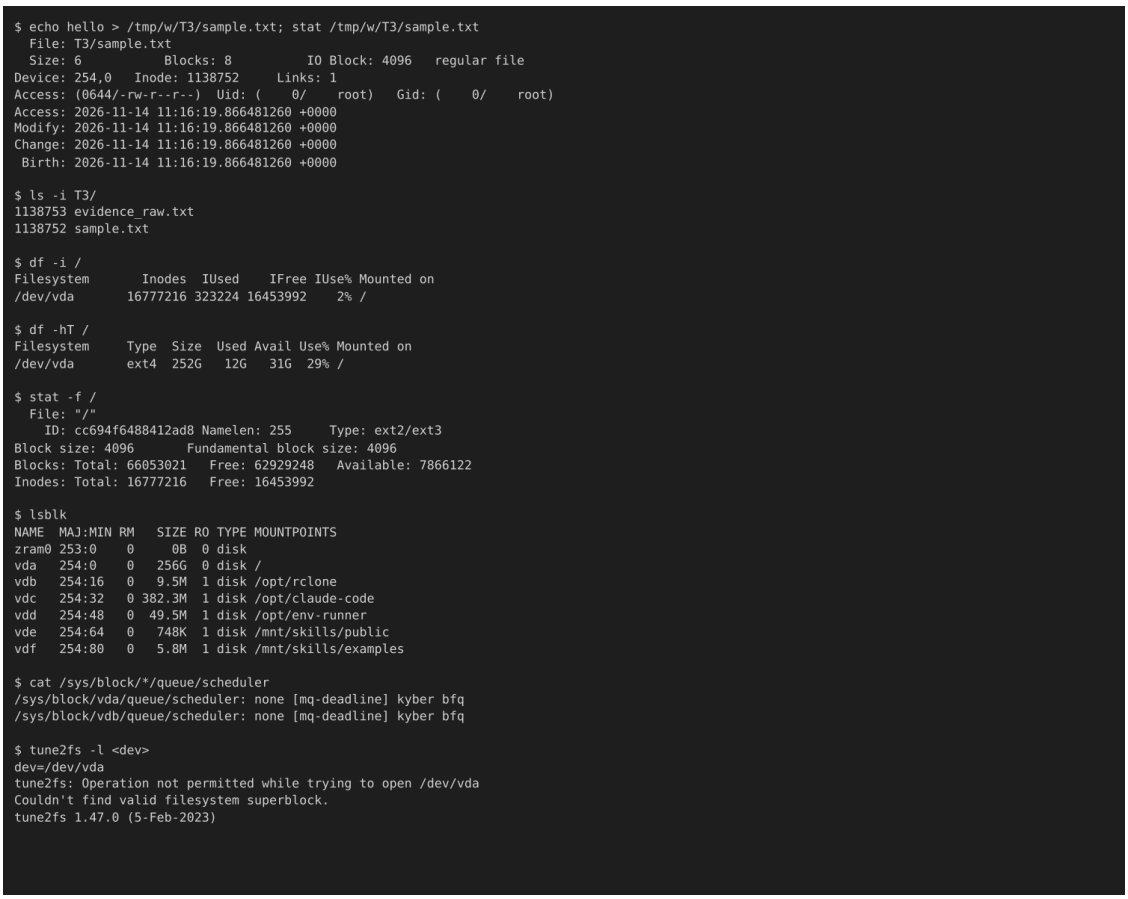


Figure 3.1 – stat, ls -i, df -i, df -hT, stat -f, lsblk and the I/O scheduler

Observation	Meaning
stat: Size 6, Blocks 8, IO Block 4096, Inode 1138752	The 6-byte file uses one 4 KiB block (8 sectors of 512 bytes); the inode number is the file's identity (same as ls -i: 1138752).
Access: (0644/-rw-r--r--) Uid 0 Gid 0	Permission bits and owner stored in the inode, not in the directory.
Links: 1	One directory entry points to this inode.
df -i /: 16,777,216 inodes, 323,224 used (2%)	Number of files is limited by the inode table; the file system can run out of inodes before space.
df -hT /: ext4, 252G, 29% used	File-system type and free space.
stat -f /: block size 4096; 66,053,021 blocks; 16,777,216 inodes	Block size 4 KiB; 66,053,021 × 4 KiB ≈ 252 GiB; about 4 blocks per inode on average (one inode per 16 KiB).

tune2fs -l / dumpe2fs -h. These commands read the ext4 superblock and need permission to open the block device. In the restricted environment used for this report the command was refused ("Operation not permitted", see Figure 3.1), so no superblock output is shown here. On your own computer run:

```
sudo tune2fs -l /dev/sda1 | egrep "Block size|Inode count|Inode size|Free blocks|Free inodes|Filesystem features|Journal"
sudo dumpe2fs -h /dev/sda1 | head -30
```

and report these fields: **Block size** (normally 4096), **Inode count**, **Inode size** (normally 256), **Free blocks**, **Free inodes**, **Filesystem features** (has_journal, extent, metadata_csum, ...), and **Journal size**. The values already obtained from stat -f (block size 4096, 66,053,021 blocks, 16,777,216 inodes) are the same quantities.

Alternative Concept and Complexity Analysis – copy-on-write file system (Btrfs/ZFS) for the database server

Solution. The data directory of the database would sit on a Btrfs subvolume or a ZFS dataset (ZFS recordsize=16K for 8–16 KB database pages; Btrfs with nodatacow or ZFS with primarycache=metadata for the data files). Every update is written to a new location and committed by a root-pointer switch, so no journal is needed; consistent snapshots of the database are taken in milliseconds.

```
ALGORITHM CoWUpdate(block, newData)              -- copy-on-write
BEGIN
   new <- AllocateFreeBlock()
   Write(new, newData)
   FOR EACH ancestor FROM parent(block) TO root
      copy the ancestor to a new block and point it at the new child
   ENDFOR
   Flush()
   WriteRootPointerAtomically()                  -- commit point
END

ALGORITHM JournaledUpdate(block, newData)        -- ext4 with journal
BEGIN
   AppendToJournal(block, newData)
   AppendCommitRecord()                          -- durable: transaction is committed
   Write(block, newData)                         -- checkpoint: in-place write
   MarkJournalEntryDone()
END
```

Criterion	ext4 with journaling	Copy-on-write (Btrfs / ZFS)
Write cost of a small random update (4 KB)	Roughly 1 data write in place + 1–2 journal writes (metadata commit) + a later checkpoint write ≈ **3 block writes**, O(1). Overwrites in place, so the layout stays sequential.	1 new data block + all changed tree nodes up to the root (tree height h ≈ 3–4) + root pointer ≈ **h + 2 writes**, O(log n) in the tree height. Writes are batched into one sequential transaction, which helps on SSD, but old and new blocks are scattered, so later sequential scans of a table become fragmented.
Extra space: snapshots	None built in (LVM snapshots need a separate area and slow writes).	Only the changed blocks are kept: a snapshot costs about the size of the changes since it was taken, O(changed blocks), plus metadata.
Extra space: checksums	Metadata checksums only (negligible).	4 bytes per 4 KB block with crc32c = 0.1%; 32 bytes with SHA-256 = 0.8%. In ZFS the checksum is stored in the parent block pointer.
Recovery after power failure	Replay of the journal: O(journal size), typically 1–5 seconds; the data may be lost but metadata is consistent.	No replay: mount uses the last committed root, O(1). An optional scrub (O(data size)) verifies the checksums.
Free-space need	Normal.	Needs headroom (about 10–20 % free) so that new blocks can always be allocated; performance drops when nearly full.
Verdict for a database	**Better for write-heavy random updates**; no fragmentation of the data files.	Better for integrity (checksums, self-healing), backups (snapshots) and replication; costs more writes and fragmentation unless tuned.

These figures are analytical estimates of block writes, not measurements: the Btrfs and ZFS mounts could not be created in the restricted environment used for this report. On your own machine the comparison can be measured with fio --rw=randwrite --bs=4k --fsync=1 *on an ext4 and a Btrfs loop-device image.*

> **Result:** XFS (or ext4 ordered) is chosen for the database server, exFAT for the pen drive, the native file system for the laptop and ZFS for the backup server. Journaling gives fast recovery by replaying a small log; copy-on-write gives always-consistent state, checksums and cheap snapshots at the cost of extra writes and fragmentation.

Task 4 – Disk Scheduling Algorithms (7 Marks)

Scenario. Cylinders 0–199, head at cylinder 53, request queue **98, 183, 37, 122, 14, 124, 65, 67**. Direction assumptions (as in the given program): SCAN and LOOK first move towards cylinder 0; C-SCAN and C-LOOK move towards the highest cylinder and then jump back.

Task 4.1 – Calculate (2 Marks)

Algorithm	Order of service	Calculation
FCFS	98, 183, 37, 122, 14, 124, 65, 67	|98−53|+|183−98|+|37−183|+|122−37|+|14−122|+|124−14|+|65−124|+|67−6 = 45+85+146+85+108+110+59+2
SSTF	65, 67, 37, 14, 98, 122, 124, 183	12+2+30+23+84+24+2+59
SCAN (to 0 first)	37, 14, (0), 65, 67, 98, 122, 124, 183	53→0 = 53; 0→183 = 183; 53 + 183
C-SCAN (up first)	65, 67, 98, 122, 124, 183, (199), jump to 0, 14, 37	53→199 = 146; 0→37 = 37; sweep total 146 + 37 = 183; the return jump 199→0 adds 199

Algorithm	Order of service	Calculation
LOOK (down first)	37, 14, 65, 67, 98, 122, 124, 183	53→14 = 39; 14→183 = 169; 39 + 169
C-LOOK (up first)	65, 67, 98, 122, 124, 183, jump to 14, 37	53→183 = 130; 14→37 = 23; sweep total 153; the return seek 183→14 adds 169

The C-SCAN value 183 and its "+199 for the jump" are the figures produced by the given program. Textbooks differ on whether the return trip is counted, so both numbers are shown; the return of the head is a single fast seek in a real drive, but it is not free.

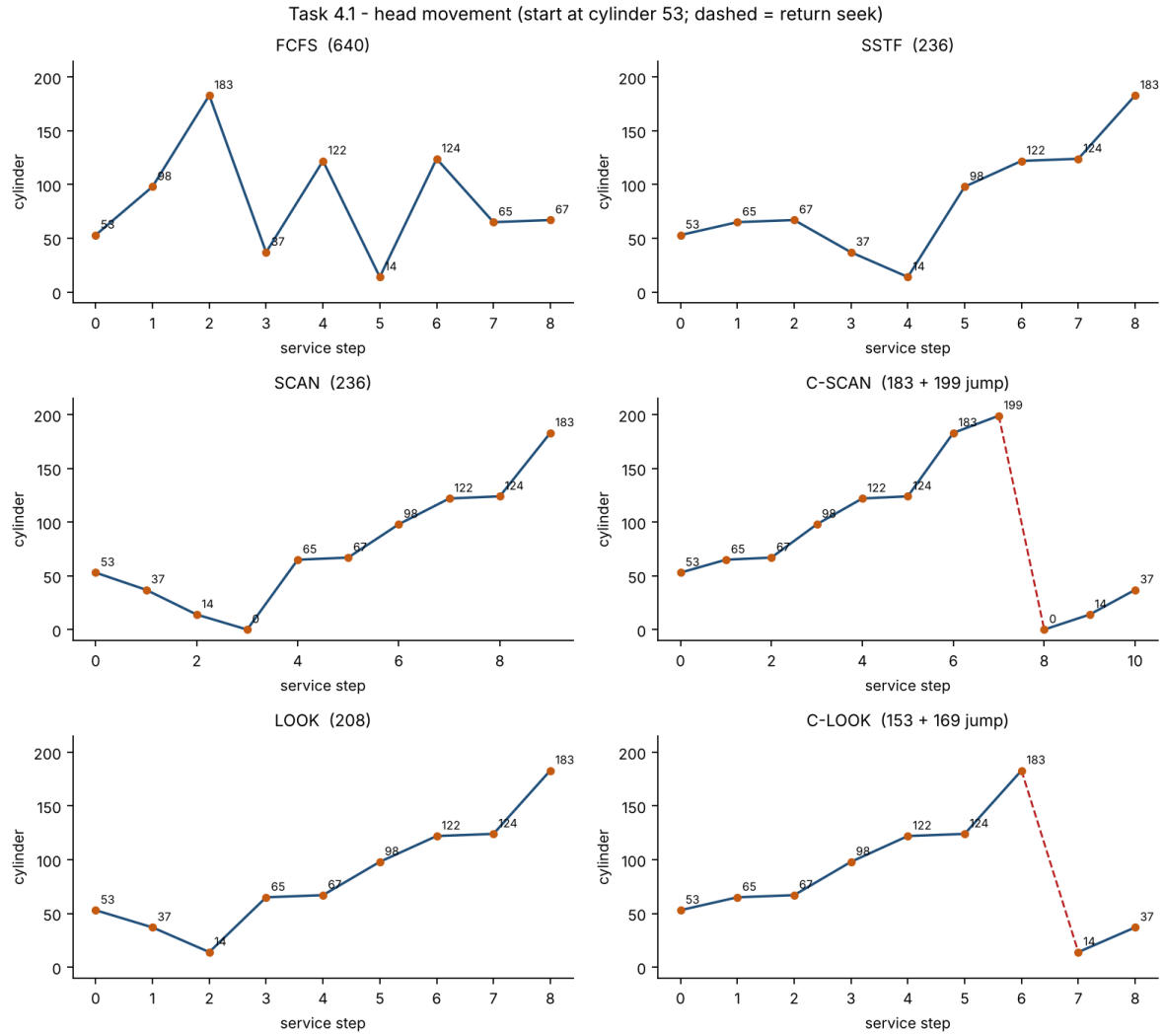


Figure 4.1 – Head movement of the six algorithms (dashed line = return seek)

Complete the Code – SSTF

TODO 1 finds, among the requests not yet done, the one with the smallest distance to the current head position, marks it done, stores it in order[k] and moves the head:

```
for (int k = 0; k < N; k++) {
    int best = -1;
    for (int i = 0; i < N; i++) {
        if (done[i]) continue;
        if (best < 0 || abs(req[i] - pos) < abs(req[best] - pos)) best = i;
    }
    done[best] = 1;
    order[k] = req[best];
    pos = req[best];
}
```

```
$ gcc -std=c11 -Wall -Wextra -o task4 task4.c
$ ./task4
FCFS   53 98 183 37 122 14 124 65 67 -> 640 cylinders
SSTF   53 65 67 37 14 98 122 124 183 -> 236 cylinders
SCAN   53 37 14 0 65 67 98 122 124 183 -> 236 cylinders
C-SCAN 53 ... 199, jump to 0 ... 37 -> 183 cylinders (+199 for the jump)
```

Figure 4.2 – Output of task4.c (identical to the expected output)

Stream of requests near cylinder 60 – what happens to cylinder 183 under SSTF

starve.c adds one new request near cylinder 60 every millisecond to the original queue. A request takes 4 ms plus 0.05 ms per cylinder to serve, so requests arrive about four times faster than they can be served and the queue grows.

```
$ gcc -std=c11 -Wall -Wextra -O2 -o starve starve.c
$ ./starve
== SSTF ==
stream   100 ms: request 183 NOT served; still waiting, 70 requests queued, head near 60
stream  1000 ms: request 183 NOT served; still waiting, 745 requests queued, head near 59
stream 10000 ms: request 183 NOT served; still waiting, 7495 requests queued, head near 58
== SCAN (up first) ==
stream   100 ms: request 183 served after 30.5 ms
stream  1000 ms: request 183 served after 30.5 ms
stream 10000 ms: request 183 served after 30.5 ms
```

Figure 4.3 – SSTF starvation test (starve.c)

Evaluation. SSTF always picks the nearest request. The new requests lie within a few cylinders of the head, whereas 183 is about 123 cylinders away, so it is **never chosen** while the stream lasts: after 100 ms, 1,000 ms and 10,000 ms it is still waiting (queues of 70, 745 and 7,495 requests), and its waiting time is at least as long as the stream. This is **starvation**. Under SCAN the same request is served after 30.5 ms because the head must sweep to the end of the disk and cannot stay near 60. A remedy for SSTF is aging (the priority of a request grows with its waiting time) or a deadline scheduler.

Task 4.2 – Simulate (2 Marks)

sim.c (Appendix A) implements all six algorithms in an event-driven simulation of **10,000 random requests** (uniform cylinders 0–199, exponential inter-arrival times, fixed seed). Service time = 4 ms (rotation + transfer) + 0.05 ms per cylinder moved; waiting time = time from arrival until service starts; head movement includes the travel to the disk end in SCAN/C-SCAN and the return seek in C-SCAN/C-LOOK. Load levels: **light** = mean 40 ms between requests (about 18% utilisation), **medium** = 9 ms (about 80%), **heavy** = 5 ms (more requests than FCFS can serve).

```
$ gcc -std=c11 -Wall -Wextra -O2 -o sim sim.c -lm
$ ./sim
load,mean_interarrival_ms,algorithm,avg_head_move_per_req,avg_wait_ms,p99_wait_ms,max_wait_ms
light,40,FCFS,66.62,0.96,12.09,28.01
light,40,SSTF,66.01,0.92,11.78,25.77
light,40,SCAN,125.25,3.39,19.58,33.24
light,40,C-SCAN,176.04,9.01,27.17,36.93
light,40,LOOK,66.02,0.92,11.93,25.77
light,40,C-LOOK,66.61,2.63,16.39,39.11
medium,9,FCFS,66.62,18.96,89.11,133.97
medium,9,SSTF,53.73,7.68,47.40,127.60
medium,9,SCAN,72.12,12.29,48.58,85.58
medium,9,C-SCAN,82.79,19.83,59.38,94.64
medium,9,LOOK,54.22,7.92,43.36,77.69
medium,9,C-LOOK,57.87,11.29,48.80,79.31
heavy,5,FCFS,66.62,11901.60,23823.12,24098.34
heavy,5,SSTF,18.00,42.72,194.50,345.53
heavy,5,SCAN,18.30,44.37,182.11,330.45
heavy,5,C-SCAN,18.55,62.58,197.03,287.17
heavy,5,LOOK,18.02,41.80,177.35,330.55
heavy,5,C-LOOK,18.44,57.92,191.79,284.27
```

Figure 4.4 – Raw simulation output (sim.c)

Load	Algorithm	Avg head movement per request (cyl.)	Avg wait (ms)	99th-percentile wait (ms)	Max wait (ms)
Light	FCFS	66.62	0.96	12.09	28.01
	SSTF	66.01	0.92	11.78	25.77
	SCAN	125.25	3.39	19.58	33.24
	C-SCAN	176.04	9.01	27.17	36.93
	LOOK	66.02	0.92	11.93	25.77
	C-LOOK	66.61	2.63	16.39	39.11
Medium	FCFS	66.62	18.96	89.11	133.97
	SSTF	53.73	7.68	47.40	127.60
	SCAN	72.12	12.29	48.58	85.58
	C-SCAN	82.79	19.83	59.38	94.64
	LOOK	54.22	7.92	43.36	**77.69**
	C-LOOK	57.87	11.29	48.80	79.31
Heavy	FCFS	66.62	11,901.60	23,823.12	24,098.34
	SSTF	18.00	42.72	194.50	345.53
	SCAN	18.30	44.37	182.11	330.45
	C-SCAN	18.55	62.58	197.03	287.17
	LOOK	18.02	41.80	177.35	330.55
	C-LOOK	18.44	57.92	191.79	**284.27**

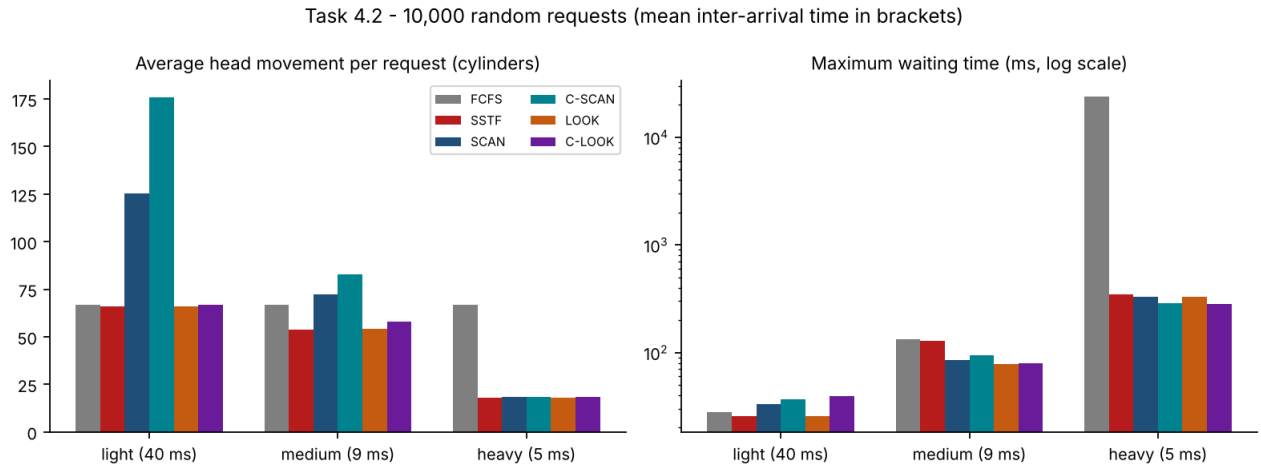


Figure 4.5 – Average head movement and maximum waiting time (log scale)

Observations

- **Light load:** the queue is almost always empty, so there is little to reorder; all algorithms are alike except that SCAN and C-SCAN waste movement travelling to the ends of the disk (125 and 176 cylinders per request against 66).
- **Medium load:** SSTF and LOOK reduce head movement by about 19% against FCFS (53.7 and 54.2 against 66.6 cylinders) and cut the average wait from 19 ms to about 8 ms. LOOK has the smallest maximum wait (77.7 ms); SSTF's maximum (127.6 ms) is much longer than its average, which is the start of starvation.
- **Heavy load:** FCFS collapses (queue grows without bound, maximum wait 24 s) because its 66-cylinder seeks cannot keep up. The elevator algorithms shorten the seeks to about 18 cylinders and stay stable. The circular algorithms C-SCAN and C-LOOK have the **lowest maximum waits (287 and 284 ms)**, against 330–346 ms for SSTF, SCAN and LOOK, although their averages are slightly higher.

Task 4.3 – Evaluate Fairness (2 Marks)

Algorithm	Can starve?	Reason
FCFS	No	Strict arrival order, so every request is served; but average wait is long under load.
SSTF	**Yes**	A stream of requests near the head keeps pushing a distant request back (shown above for cylinder 183). Worst for the cylinders at the edges.
SCAN, C-SCAN	No (waiting time bounded)	Every request is served within one or two sweeps of the arm. Plain SCAN can keep serving new requests at the head's own cylinder; variants such as N-step SCAN and FSCAN freeze the queue during a sweep to prevent this.

Algorithm	Can starve?	Reason
LOOK, C-LOOK	No (bounded)	Same bound as SCAN/C-SCAN, with shorter sweeps.

Why C-SCAN gives a more uniform waiting time than SCAN

Let a full one-way sweep take T and let p (from 0 to 1) be the position of a cylinder along the disk. **SCAN** visits a cylinder on the way up and again on the way down. The gaps between its visits are $2(1-p)T$ and $2pT$. A request arriving at a random time therefore waits on average $T\cdot[(1-p)^2 + p^2]$: about **T at the edges** ($p = 0$ or 1) and only **T/2 in the middle**. Cylinders at the edges have to wait up to 2T, while the cylinders in the middle are visited almost twice as often. **C-SCAN** moves in one direction only and returns without serving, so every cylinder is visited exactly once per cycle of length $T_c \approx T$ + return time. The expected wait is $T_c/2$ for **every** cylinder and the maximum is T_c, so waiting time is the same for every position. This is confirmed by the simulation: under heavy load C-SCAN's maximum wait (287 ms) is lower than SCAN's (330 ms), even though its average is higher.

Task 4.4 – Solid-State Drives (1 Mark)

Why "none" or "mq-deadline" are used for SSDs. The scheduler algorithms above exist to reduce seek time and rotational delay. An SSD has neither: every block is reached in about the same time (tens of microseconds), so reordering requests saves nothing and only costs CPU time and latency. An NVMe SSD also has many hardware queues and internal parallelism and runs best when requests are passed straight to the device: the Linux multi-queue block layer with **none** adds no scheduling at all. **mq-deadline** is a light-weight scheduler used for SATA SSDs and virtual disks: it only merges adjacent requests and gives each request a deadline (reads 500 ms, writes 5 s by default), so reads are not starved by writes and latency stays bounded, without any seek optimisation.

Check on the real system. (Figure 3.1, taken from /sys/block/<disk>/queue/scheduler):

```
/sys/block/vda/queue/scheduler: none [mq-deadline] kyber bfq
/sys/block/vdb/queue/scheduler: none [mq-deadline] kyber bfq
/sys/block/loop0/queue/scheduler: [none] mq-deadline kyber bfq
```

The scheduler shown in brackets is the active one: the virtual disk vda (the root file system) uses **mq-deadline**, and the loop devices use **none**. On your own machine run cat /sys/block/sda/queue/scheduler (or nvme0n1) and note the result; to change it: echo none | sudo tee /sys/block/sda/queue/scheduler.

Alternative Concept and Complexity Analysis – LOOK and C-LOOK

Solution. LOOK and C-LOOK follow the same sweep as SCAN and C-SCAN but turn round at the **last request** in the direction of travel instead of going to the end of the disk. The scheduler must therefore know where the last request is.

```
ALGORITHM LOOK(head, queue, direction)
INPUT  head position, set of pending cylinders, direction (UP or DOWN)
OUTPUT service order
BEGIN
   WHILE queue is not empty
      ahead <- { r IN queue : r is in direction of travel from head }
      IF ahead is empty THEN
         direction <- opposite(direction)          -- turn at the last request
      ELSE
         r <- member of ahead nearest to head      -- O(n) if unsorted, O(log n) if
sorted
         Serve(r); remove r from queue; head <- r
      ENDIF
   ENDWHILE
END

ALGORITHM CLOOK(head, queue)                       -- always moves UP
BEGIN
   WHILE queue is not empty
      IF there is a request >= head THEN serve the nearest one above head
      ELSE head <- smallest request in queue       -- jump back to the lowest
pending request
      ENDIF
   ENDWHILE
END
```

Quantity	SCAN	LOOK	C-SCAN	C-LOOK	Remark
Head movement for the given queue (sweep only)	236	**208** (−11.9%)	183	**153** (−16.4%)	Given queue
Head movement incl. return jump	236	208	382	**322** (−15.7%)	Return seek counted
Average head move / request, medium load (simulated)	72.12	**54.22**	82.79	**57.87**	10,000 requests

Quantity	SCAN	LOOK	C-SCAN	C-LOOK	Remark
Average wait, medium load (ms)	12.29	**7.92**	19.83	**11.29**	Simulated
Maximum wait, medium load (ms)	85.58	**77.69**	94.64	**79.31**	Simulated
Maximum wait, heavy load (ms)	330.45	330.55	287.17	**284.27**	Simulated

Extra work and complexity. SCAN and C-SCAN need no knowledge of the queue: the head simply runs to the end, so choosing the next request costs O(1) with a sorted queue. LOOK and C-LOOK must find the last request in the direction of travel: with an **unsorted** list this is an O(n) scan per decision ($O(n^2)$ to serve n requests); with a **sorted** queue (balanced tree or sorted array) the nearest request in either direction and the last request are found in O(log n) per lookup, and inserting a new request costs O(log n) (O(n) for an array). Space is O(n) in all cases. LOOK/C-LOOK therefore trade a little bookkeeping for shorter sweeps; they save the idle travel to the disk ends, which is why Linux's elevator-style schedulers work this way.

> **Result:** FCFS 640, SSTF 236, SCAN 236, C-SCAN 183 (+199 jump), LOOK 208 and C-LOOK 153 (+169 return) cylinders for the given queue; the programs reproduce the expected output. In 10,000-request simulations LOOK/SSTF reduce movement by about 19% at medium load, FCFS collapses at heavy load, and C-SCAN/C-LOOK give the lowest maximum waits. SSTF starves the request at cylinder 183 when requests keep arriving near 60. SSDs use none or mq-deadline because they have no seek time.

Task 5 – Distributed and Cloud-Based Storage (7 Marks)

Scenario. A university wants all student and staff files to be available from any campus and from home.

Task 5.1 – Compare the Options (3 Marks)

Criterion	NFS (one file server)	Distributed FS: HDFS / Ceph	Cloud object storage: S3 / GCS
Consistency	Close-to-open consistency: a client sees another client's changes after the file is closed and re-opened; clients cache attributes and data, so short-term stale reads are possible. NFSv4 adds locks and delegations.	HDFS: strong for namespace operations (NameNode), files are write-once/append-only with a single writer. Ceph (RADOS): strongly consistent, writes acknowledged after all replicas have them.	Strong read-after-write and list consistency for single objects on S3 and GCS; no partial updates (an object is replaced as a whole); no POSIX locking or rename.
Availability	One server is a single point of failure; HA pairs or clusters are possible but complex. Expected about 99.5% on commodity hardware.	Replicated blocks (3 copies by default) across racks; NameNode HA; survives disk, node and rack failures; self-healing.	Designed for very high durability (11 nines) and about 99.9–99.99% availability by SLA, replicated across zones; the provider runs it.
Performance	Good for small files on a LAN with low latency; limited by one server's disk and network; slows as users grow (see the alternative analysis).	High aggregate throughput that grows with the number of nodes; HDFS is poor for many small files (NameNode memory) and for random writes; Ceph handles both with more tuning.	High throughput for large objects, but high first-byte latency (tens of ms) and per-request overhead over the WAN; not suited to many small random updates.

Criterion	NFS (one file server)	Distributed FS: HDFS / Ceph	Cloud object storage: S3 / GCS
Cost	Lowest at small scale: one server and a disk array; staff time for administration and backups.	Commodity hardware but 3× raw storage (1.5× with erasure coding) and skilled staff to run it.	No hardware; pay per GB-month, per request and per GB of data leaving the cloud; cost grows with use and egress.
Best fit	Single campus, home directories, small scale.	Large on-premise data, analytics (HDFS), shared POSIX storage for many campuses (CephFS).	Archives, backup copies, access from anywhere, web delivery.

Recommendation: a replicated distributed file system (CephFS) on campus for the daily POSIX home directories, an object-storage gateway or sync client for access from home, and cloud object storage for off-site backups. NFS alone cannot give availability across campuses; cloud-only gives up control and adds cost for every download.

Task 5.2 – Replication and Consistency (2 Marks)

How replication protects data (HDFS, 3 copies)

- **Placement:** a file is cut into 128 MB blocks; each block is stored on 3 DataNodes: the first copy on the writer's node, the second on a node in a different rack and the third on another node in that second rack. This survives the failure of a disk, a node or a whole rack.
- **Detection and repair:** DataNodes send heartbeats (every 3 seconds) to the NameNode; when one is silent for about 10 minutes its blocks are "under-replicated" and the NameNode orders new copies to be made from the surviving ones, restoring 3 copies automatically.
- **Reads:** a client reads from the nearest replica, so replication also spreads the read load and increases throughput.
- **Cost:** 3× raw storage (200% overhead); erasure coding (e.g. 6 data + 3 parity blocks) cuts this to 1.5× at the price of more CPU and network work on reads after a failure.

Strong against eventual consistency

	Strong consistency	Eventual consistency
Meaning	After a write completes, every later read returns it, wherever it is served from.	If no new writes arrive, all replicas will finally hold the same value; meanwhile reads may return old data.
How	Write to all replicas (or a quorum with R + W > N, e.g. N = 3, W = 2, R = 2) before acknowledging.	Acknowledge after one replica; propagate in the background (anti-entropy, read repair).
Trade-off	Higher write latency; during a network partition the system may refuse requests (consistency over availability).	Low latency and always available, but the application must tolerate stale reads and conflicts (availability over consistency).
Use for	Bank balances, file-system metadata, course marks.	Shopping carts, "likes", cached copies, DNS.

Task 5.3 – Device Management (2 Marks)

```
read(fd, buf, n)      application
  |  system call
VFS / file system     finds the file block (inode, extent)
  |
page cache            HIT -> copy from memory, done        (caching)
  | MISS
block layer           merge adjacent requests, I/O scheduler (none / mq-deadline)
(buffering)
  |
device driver         builds the command for the controller (SATA, NVMe)
  |
controller + DMA      transfers data to memory without the CPU
  |  interrupt
driver completion     wakes up the process; data is placed in the page cache
```

Concept	Explanation
Device driver	Kernel module that hides the hardware details: it translates generic requests (read block 100) into the commands of one controller and handles its interrupts. A new disk needs only a new driver, not changes to the file system.

Concept	Explanation
Block-device interface	Disks appear as arrays of fixed-size blocks (for example /dev/sda, 4 KB) that can be read or written at any block number (random access); the file system and the I/O scheduler work only through this interface. Character devices (keyboard, serial port) give a stream of bytes instead.
Buffering	A memory area that holds data while it is moved between two parts with different speeds or sizes, for example between a fast process and a slow disk, or to adapt block size. Double buffering lets the next block be read while the previous one is processed.
Caching	A copy of recently or frequently used disk blocks kept in RAM (the page cache), so later accesses do not touch the disk; read-ahead and write-back make sequential access and bursts of writes cheaper. Unlike a buffer, a cache holds a copy for faster re-use.
Spooling	Data for a device that can serve only one job at a time (a printer) is first stored in a queue on disk; a daemon sends it to the device while the program continues. It allows many users to "print" at the same time without waiting.

Alternative Concept and Complexity Analysis – one large NFS server

Solution. All files would live on a single NFS server (with RAID inside it); every campus and home user mounts server:/export/home. Every read and write is served by that one machine.

```
ALGORITHM ReadFile_NFS(file, offset, n)
    server <- THE_ONE_SERVER                    -- same machine for all
    RETURN server.Read(file, offset, n)         -- queueing delay grows

ALGORITHM ReadFile_Replicated(file, offset, n)
    locations <- NameNode.GetReplicas(file, offset)   -- metadata lookup
    replica   <- NearestAvailable(locations)          -- spreads load, skips failures
    RETURN replica.Read(file, offset, n)
```

Analytical model (M/M/1 queue, **calculated**, not measured): the server handles μ = 1,000 requests/s; each user issues 2 requests/s. For one server the response time is $R = 1/(\mu - \lambda)$. For three replicas serving reads, each gets $\lambda/3$.

Users	Load on one NFS server λ (req/s)	NFS response time R (ms)	Load per replica ($\lambda/3$)	Replicated response time (ms)
250	500 ($\rho = 0.50$)	2.0	167	1.2

Users	Load on one NFS server λ (req/s)	NFS response time R (ms)	Load per replica ($\lambda/3$)	Replicated response time (ms)
450	900 ($\rho = 0.90$)	10.0	300	1.4
490	980 ($\rho = 0.98$)	50.0	327	1.5
500 or more	$\geq$ 1,000 ($\rho \geq 1$): saturated	unbounded	333+	1.5
1,000	2,000: saturated	unbounded	667 ($\rho = 0.67$)	3.0

Criterion	One large NFS server	Replicated distributed file system
Read time as users grow	Time rises as $1/(\mu - \lambda)$ and grows without bound near saturation: **$O(1/(1-\rho))$**. Capacity is fixed by one server's disks and network card.	Reads are spread over replicas, so capacity grows with the number of nodes; time stays low until each node nears its own limit.
Write time as users grow	Single server: same queueing growth, but the write completes after one write (plus RAID).	Each write must reach all 3 replicas (pipeline), so write time is slightly longer (network hops) and **writes do not scale with replicas**: every replica takes every write.
Storage used	1 copy; with RAID-6 about 1.2×; for 100 TB usable ≈ 120 TB raw (+ backup).	3 replicas: **300 TB raw** for 100 TB usable (3×); 150 TB with 6+3 erasure coding.
Availability with one server failed	Assume 99.5% availability: 0.5% × 8,760 h ≈ **44 hours/year** of downtime; when the server fails, all users lose access.	Service continues from the other copies; the chance that all 3 independent servers are down together is $0.005^3 = 1.25 \times 10^{-7}$ ≈ **4 seconds/year** (idealised: ignores failover time and correlated failures).
Complexity	Simple to run.	Needs metadata servers, placement, repair: more complex to operate.

> **Result:** A single NFS server is cheapest and simplest, but its response time grows without bound as the number of users approaches its capacity and its failure stops everyone. A replicated distributed file system pays 3× storage and more complexity for scalable reads and survival of server failures; cloud object storage adds off-site durability and access from anywhere at the price of latency and recurring cost.

Task 6 – Design Challenge: Video-Streaming Start-up (13 Marks)

Scenario. 200,000 students; each video is 300 MB – 2 GB, uploaded once, read many thousands of times (mostly in the first week); students jump to any minute. A few servers with hard disks and SSDs, plus optional cloud storage.

Stage 1 – Interpret the Workload (2 Marks)

Operations and access patterns

Operation	Frequency / pattern	Consequence for the design
Upload (write once)	Rare: a few videos per day; sequential write of 0.3–2 GB	Write path can be simple; allocate each file in large extents (fallocate)
Start playback at any minute	Peak about 28 starts per second (about 100,000 per hour in the first week); **random seek**, then sequential	First chunk must come quickly: latency is the key target
Continue playback	Constant: each stream reads 2.5 MB about every 6.7 s; sequential	Throughput and read-ahead
Popularity	Strongly skewed (a few videos draw most views – Zipf-like), mostly in the first week	Caching pays off hugely; popular videos in RAM/SSD
Delete / replace	Rare	No need to optimise

Assumptions used for all calculations

- Catalogue of **3,000 videos**, sizes uniform between 0.3 and 2.0 GB: mean 1.16 GB, total **3.39 TB** (simulated).
- Video bit-rate **3 Mbit/s** = 0.375 MB/s; **peak 4,000 concurrent viewers** (2% of the 200,000 students); read unit (chunk) 2.5 MB ≈ 6.7 s of video.
- Popularity follows Zipf (exponent 1): the k-th most popular video has probability proportional to 1/k.

Requirements stated as testable statements

ID	Requirement	How it is tested
R1	At peak (4,000 streams) the first 2.5 MB chunk of any minute of any video is delivered from storage in ≤ 100 ms for 95% of starts (end-to-end start of playback ≤ 2 s).	Simulation (Stage 5); fio latency percentiles
R2	Sustain 4,000 × 3 Mbit/s = 12 Gbit/s with no stream stalling for more than 1% of its time.	Throughput test with 600 chunk reads/s
R3	Locating any minute needs ≤ 2 disk reads when metadata is not cached and 1 when it is.	Count reads with blktrace or the calculation in Stage 4
R4	No hard disk may be more than 70% busy at peak.	Measured utilisation (Stage 5)
R5	The loss of any one disk or server causes no data loss and no outage.	Fail a disk in the test; check replicas
R6	A newly uploaded video can be watched within 10 minutes.	Upload test

Stage 2 – Propose the Storage Architecture (2 Marks)

Layer	Proposal
Hot tier – RAM (page cache)	About 64 GB across the cluster: the few dozen most-watched videos stay in the page cache (about 53% of all reads in the simulation).
Warm tier – NVMe SSD	About 2 TB across the cluster, holding the next most popular videos (about 1,700 videos, 40% of reads). Used as a cache (dm-cache / bcache / lvmcache) or as a pinned tier, so no replica is needed.
Cold tier – hard disks	12 HDDs of 4 TB (4 per server on 3 servers). Every video is stored **twice** on different servers: 3.39 TB × 2 = 6.8 TB. Hard disks give the cheapest capacity for the large rarely-watched tail.
Cloud object storage	The "golden copy" of every video (3.4 TB) and the backup: used as the source from which a lost replica or a new server is rebuilt. Not in the playback path.
File system	**XFS** with 4 KB blocks: built for large files and parallel I/O; large extents (a 2 GB file needs 16 extents of 128 MB); metadata journal for fast recovery.

Layer	Proposal
Allocation method	**Extent-based indexed allocation**: each upload is pre-allocated with fallocate in one or a few long extents, which gives almost contiguous files for fast sequential reads, and O(1) location of any minute through the extent tree.

Stage 3 – Identify the Critical Design Decisions (2 Marks)

Decision	Choice	Reason
Block size	4 KB file-system block, 2.5 MB read-ahead (chunk)	4 KB equals the page size (the Linux limit on common hardware) and wastes at most 4 KB per file (0.0004% of a 1 GB file). Because videos are large and read sequentially, throughput comes from the **big read unit and extents**, not from big blocks. A 2.5 MB read is 640 contiguous blocks served in one I/O.
Disk scheduler	HDD: mq-deadline; NVMe SSD: none	On hard disks the elevator logic and merging reduce seeks (Task 4: 18 against 66 cylinders per request under load) and the deadline keeps reads from starving; the NVMe drive has no seek time, so reordering would only add delay.
Caching of popular videos	RAM page cache + SSD tier, filled by popularity counters (promote after 2 views in an hour, evict the least recently used)	Zipf popularity: 53% of reads come from RAM and 94% from RAM + SSD, so only 6% of reads reach the HDDs (simulated).
Replication factor	**2** copies on HDD (different servers) + 1 golden copy in the cloud	Videos are immutable and re-creatable from the cloud, so a third on-disk copy is not needed; 2 copies give read balancing (shorter queue is chosen) and survive one failure. The cost is 2× storage (6.8 TB).

Stage 4 – Design and Calculate (4 Marks)

(a) Capacity

```
videos          : 3,000 x 1.16 GB (mean)      = 3.39 TB
HDD storage     : 3.39 TB x 2 replicas        = 6.8 TB   (of 48 TB: 14% used)
SSD tier        : 2 TB  (about 1,700 most popular videos)
```

```
RAM cache         : 64 GB (about 55 most popular videos)
cloud golden copy : 3.39 TB (3,390 GB x $0.023 = about $78/month, assumed price)
```

(b) Throughput

```
stream rate       : 3 Mbit/s = 0.375 MB/s
peak aggregate    : 4,000 x 0.375 MB/s = 1,500 MB/s = 12 Gbit/s
per server (3)    : 4 Gbit/s  -> 2 x 10 GbE (or 1 x 25 GbE) network card each
chunk reads       : 4,000 x 0.375 / 2.5 MB = 600 reads/s  (+ 28 starts/s = 628 reads/s)

HDD service time  : 6.0 ms seek (elevator-reduced) + 4.17 ms rotation + 2.5 MB / 160 MB/s (15.6 ms)
                    = 25.8 ms  ->  38.8 reads/s per disk  ->  12 disks = 465 reads/s
Design A (RAM + HDD) : 47% of 628 = 293 reads/s reach the HDDs -> utilisation 293/465 = 63%
Design B (+ SSD tier): 6.3% of 628 =  40 reads/s reach the HDDs -> utilisation 40/465  = 8.5%
```

Both values match the simulated utilisation (63.0% and 8.5%). HDDs alone could not serve the 600 reads/s (they would be at 600/465 = 129% busy), which is why the RAM and SSD tiers are essential.

(c) Disk reads needed to start playing at any minute

Largest video: 2 GB = 524,288 blocks of 4 KB. A jump to minute m of a 60-minute video means offset = (m/60) × 2 GB, so block number up to 524,287.

Metadata structure	Reads (cold: nothing cached)	Reads (inode cached)
Classic block map (12 direct, single, double, triple)	Block 524,287 lies in the double-indirect range: inode + double-indirect block + single-indirect block + data = **4**	3
XFS/ext4 extent tree (chosen)	A 2 GB file has 16 extents of 128 MB. XFS keeps 4–6 extents in the inode and the rest in a B+tree leaf: inode + 1 leaf + data = **3**; with the leaf cached = 2	**1** (data only, with extent cached in memory)
Contiguous file (ideal)	directory/inode + data	1

Time to first chunk when it is not cached: 2 reads on a hard disk, about 2 × 10.2 ms (seek + rotation) + 15.6 ms transfer ≈ 36 ms; on NVMe about 1 ms. Both are well inside the 100 ms target R1 for a single request; the queueing delay under load is what Stage 5 measures.

Stage 5 – Implement and Demonstrate (3 Marks)

task6.c (Appendix A) simulates a 300-second window of the peak hour: 4,000 viewers reading 2.5 MB chunks every 6.67 s (about 188,000 chunk reads) plus 28 new starts per second at random minutes (8,400 starts). Videos have Zipf popularity; a read is served from RAM, from the SSD tier or from one of two HDD replicas (the one with the shorter queue); each device is a queue with the service times above. The two alternatives are:

- **Design A:** 64 GB RAM + 12 HDDs (no SSD tier).
- **Design B:** 64 GB RAM + 2 TB NVMe SSD tier (2 devices) + 12 HDDs as back-end.

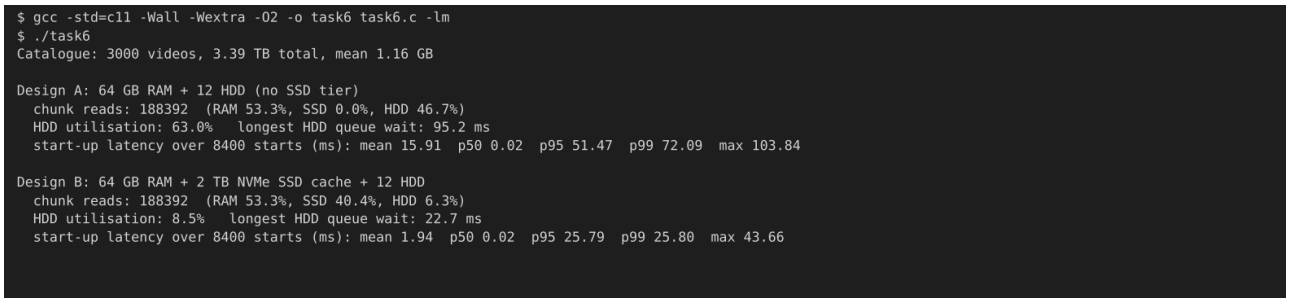


Figure 6.1 – Output of task6.c

Measured result	Design A: RAM + HDD	Design B: RAM + SSD + HDD
Reads served from RAM / SSD / HDD	53.3% / 0% / 46.7%	53.3% / 40.4% / 6.3%
HDD utilisation (limit R4 = 70%)	63.0%	**8.5%**
Longest wait in an HDD queue	95.2 ms	22.7 ms
Start-up latency: mean	15.91 ms	**1.94 ms**
Start-up latency: 95th percentile (R1: ≤ 100 ms)	51.47 ms	**25.79 ms**
Start-up latency: 99th percentile	72.09 ms	25.80 ms
Start-up latency: maximum	103.84 ms (misses R1 only at the extreme)	43.66 ms

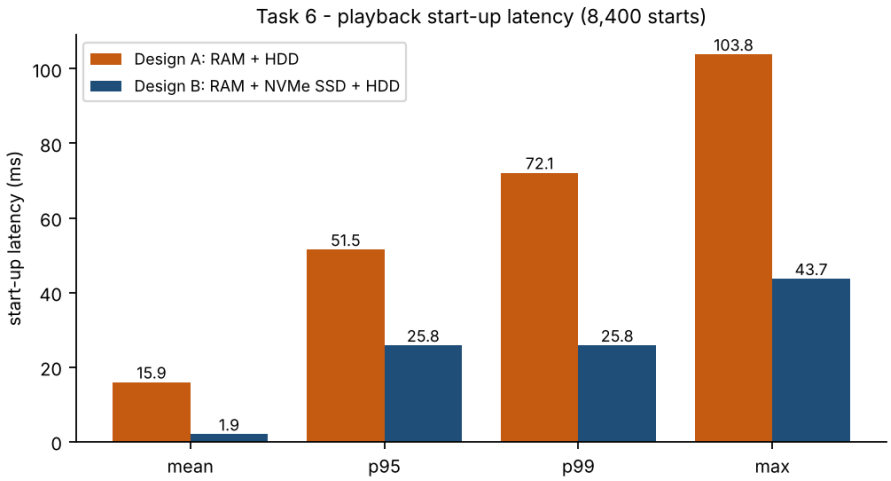


Figure 6.2 – Start-up latency of the two designs

Evidence-based choice: Design B. The SSD tier reduces the mean start-up latency by 88% (15.9 to 1.9 ms), the 95th percentile by 50% and the worst case by 58%, and it lowers the HDD load from 63% to 8.5%, leaving a large reserve for the first-week surge and for re-replication after a disk failure. Design A meets R1 at this load, but with only 7% spare disk capacity at the 70% limit it would break down if viewers or the popularity skew changed, since queueing delay grows as $1/(1-\rho)$. The extra cost of Design B is 2 TB of NVMe storage, small compared with the cost of more servers.

Validation with fio (not available in the environment used for this report; run it on your own machine to confirm the HDD and SSD service times used in the simulation):

```
; video.fio  -  mixed random-start / sequential 2.5 MB reads, 64 streams
[global]
ioengine=libaio
direct=1
rw=randread
bs=2560k
size=8g
runtime=60
time_based
group_reporting
[hdd-tier]
filename=/mnt/hdd/testfile
numjobs=16
iodepth=4
[ssd-tier]
filename=/mnt/nvme/testfile
numjobs=16
iodepth=4
; run: fio video.fio   -> read clat percentiles (p95, p99) and MB/s for each tier
```

Alternative Concept and Complexity Analysis – cloud object storage plus CDN

Solution. Every video is uploaded once to cloud object storage (S3 / GCS); a content-delivery network (CDN) with edge caches in many cities serves the viewers. The company runs no storage servers: the video is split into small segments (2–10 s) so a jump to minute m is just an HTTP range request for segment m.

```
ALGORITHM StartPlayback_CDN(video, minute)
BEGIN
   edge <- DNS gives the nearest CDN edge for the student's city
   seg  <- SegmentNumber(video, minute)
   IF edge.cache has seg THEN RETURN edge.cache[seg]          -- hit: RTT to edge
only
   ELSE
      data <- ObjectStore.GET(seg)                            -- miss: extra trip to origin
      edge.cache[seg] <- data; RETURN data
   ENDIF
END
```

Time to start playback in different cities

Model: start-up ≈ 4 × RTT (DNS, TCP, TLS, request) + time to download the first 1 MB at 25 Mbit/s (320 ms) + storage latency (26 ms for a hard disk in Design B's worst case; 5 ms for a CDN cache hit). The round-trip times are **assumed typical values**, to be replaced by measurements (ping, curl -w) from your own network.

Student's location	RTT to own server (ms)	RTT to CDN edge (ms)	Start-up, own server (ms)	Start-up, CDN hit (ms)
Same city as the data centre	10	10	4×10 + 320 + 26 = **386**	4×10 + 320 + 5 = **365**
Another city in the country	40	15	4×40 + 320 + 26 = **506**	4×15 + 320 + 5 = **385**
Abroad / remote area	180	25	4×180 + 320 + 26 = **1,066**	4×25 + 320 + 5 = **425**

A CDN cache miss adds the round trip to the object store plus its first-byte latency (about +100–150 ms), but only the first viewer of a segment in a city pays it. Because the first-week traffic is concentrated on popular videos, the hit rate is very high.

Cost per month (illustrative – prices are assumptions to be checked on the provider's price list)

```
monthly viewing  : 200,000 students x 5 h/month x 1.35 GB/h (3 Mbit/s) =
1,350,000 GB (1.35 PB)
average rate     : 1,350,000 GB x 8 / (30 x 86,400 s)          = 4.2 Gbit/s  (peak 12
Gbit/s)
```

storage : 3,390 GB x $0.023 / GB-month = about $78 / month
CDN transfer : 1,350,000 GB x $0.02-0.05 / GB = $27,000 to $67,500 / month
own servers : fixed cost (3 servers, SSD, 12 HDD, 12 Gbit/s link), not per GB

Criterion	Own servers (Design B)	Object storage + CDN
Time to start playback	386 ms nearby but 1,066 ms far away	365–425 ms in all cities: **near-uniform**
Monthly cost	Fixed (hardware depreciation, power, a 12 Gbit/s link, staff); independent of volume; sized for the first-week peak, idle later	Storage is cheap (about $78) but **transfer dominates** and grows with each viewer-hour; no cost for idle capacity
Effort of running	High: hardware, disk replacement, patching, tiering, monitoring, capacity planning	Low: provider scales and repairs; remaining work is encoding, segmenting and the billing alarms
Control and risk	Full control; single data-centre risk unless replicated	Lock-in, egress price changes, less control over data location
Complexity	Storage per node O(videos × replicas); request time O(1) with extents	O(1) per segment; cache hit rate decides cost

Result: Recommended: Design B (RAM + NVMe tier + replicated HDD with XFS extents) as the origin, giving a measured mean start-up latency of 1.9 ms and 8.5% disk load at peak, with the cloud as golden copy and, for distant cities and first-week surges, a CDN in front. This hybrid keeps the monthly bill near the fixed cost of the servers while avoiding poor start-up times for far-away students.

, adjust the pseudocode keywords to match it.